

```
$ docker images
REPOSITORY          TAG          IMAGE ID       CREATED        SIZE
glarimy/ums-add-service latest       7cd1a919486b  44 minutes ago 764MB
<none>              <none>      64288c003081  47 minutes ago 635MB
<none>              <none>      cb7ec9732570  About an hour ago 764MB
maven                3.5-jdk-8   985f3637ded4  3 years ago   635MB
```

Figure 3: Listing the Docker images

```
$ docker container ps
CONTAINER ID   IMAGE          NAMES                COMMAND                CREATED        STATUS
d3849b745533   glarimy/ums-add-service  ums-add-service      "java -jar /usr/glar..." 13 seconds ago Up 13 seconds
0.0.0.0:8080->8080/tcp
```

Figure 4: Listing the containers

for it to be deployed on the target infrastructure, the image needs to be distributed. The following command publishes the local image on the Docker Hub so that anyone can pull it for deployment. Of course, an account needs to be created on <https://hub.docker.com> before pushing the image to the hub.

```
docker push glarimy/ums-add-service
```

The images on the Docker Hub can be searched online at <https://hub.docker.com/>. Individuals and organisations can create private spaces to ensure the confidentiality of the images. In fact, automated builds can also be set up on Docker Hub in such a way that it listens to the code commits on Git and kicks off building a new version of the image immediately. Organisations may use several other tools like Jenkins and design a CI pipeline.

One way or another, once an image is available in the hub, it can be pulled to any Docker machine with the following command:

```
docker pull glarimy/ums-add-service
```

And the following command launches a new Docker container from the image and exposes it to the clients:

```
docker container run -d --name ums-add-service -p 8080:8080 glarimy/ums-add-service
```

This command may look a bit complicated, but it's not really so once it is understood.

- The `docker container run` is the command that runs the container.
- The `-d` switch makes it a daemon process so that the prompt is back.
- The `-name` switch names the container as expected.

Without this switch, the Docker Engine generates a name that may not be intuitive.

- The `-p` switch maps the port 8080 of the container to the port 8080 of the host machine so that the clients outside of the container can also access the service.
- And the last part is the actual name of the image that needs to be run, which is `glarimy/ums-add-service` in this case.
- The list of containers running currently can be found using the following command:

```
docker container ps
```

Now the service can be accessed from the host machine using the appropriate CURL command or using any REST client, just like it was done in the previous parts of this series.

Next steps

Though the Docker containers solve the problem of building ready-to-run self-contained images that can be deployed onto any Docker machine, this is certainly not sufficient. What if microservices running in two different containers want to access each other? What if a microservice wants to access a database server? What if N number of instances of a given service must always be run to handle the load?

The answer lies in networking and orchestrating the containers, which will be covered in the next part of this series. **END** 

 By: Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.